Sahaya Accessibility Platform

Design Report | July 2026

Executive summary

Sahaya is a Next.js accessibility companion designed around wheelchair users' everyday planning, navigation, reporting, and emergency-support needs. The current prototype combines accessible route planning, indoor navigation, building-image analysis, community reports, emergency SOS, and an accessibility-focused assistant.

The design prioritizes clear actions, mobile-friendly controls, server-side integrations, and cautious communication. It is a college-project prototype: users should verify physical accessibility conditions and use local emergency services for immediate danger.

At a glance

Six feature areas are available from the authenticated Sahaya dashboard: accessible routes, indoor navigation, scanner, community reports, emergency SOS, and health/assistant support.

Firebase supports authentication and data storage; OpenStreetMap/Leaflet, GraphHopper, Twilio, Google Places, and OpenAI are kept behind server-side routes where applicable.

The production build completed successfully after source files were normalized to UTF-8 and the generated build cache was recreated.

Introduction

Wheelchair users often need more than a shortest-path map: they need context about ramps, lifts, entrances, washrooms, obstacles, and local reports. Sahaya groups these needs into one accessible web application with a concise, feature-led dashboard.

The project deliberately separates deterministic systems from AI. Maps, routing, GPS, authentication, Firebase storage, and SMS delivery remain conventional service integrations. AI is reserved for language and image interpretation that benefits from reasoning.

Key findings

The current implementation has a clear client/server boundary. Browser pages call local API routes; server routes hold service credentials and return focused response data. This reduces accidental key exposure and makes each feature easier to test independently.

Context and conditions

Authentication is the entry point to application features. The login screen is limited to credentials and recovery links, while protected feature pages redirect unauthenticated visitors to login. The root page becomes the authenticated feature dashboard after sign-in.

Patterns in the evidence

Accessibility work is strongest when information is concise and qualified. The scanner reports only visible evidence, community reports retain the user's meaning, and emergency guidance avoids claiming that an exit is safe or available without verified data.

Key takeaway. Sahaya is most credible when it helps users interpret evidence and plan next actions without presenting uncertain accessibility information as a guarantee.

Theme	Observation	Implication
Access	Protected routes and focused dashboard	Keep authentication simple and visible
Evidence	Scanner and reports qualify uncertain conditions	Avoid unsupported accessibility claims
AI	Server-only Responses API usage	Reserve calls for interpretation and language

Implications

A production deployment should treat data quality as a shared responsibility. Community reports need moderation, building scans need visible-evidence limitations, and route recommendations need local verification. The interface should make those limits easy to understand without making the product feel alarmist.

Recommendations

Prioritize a small, reliable demonstration path: authenticate, plan a route, upload a clear entrance photo, submit a structured community report, and request assistant guidance. This gives reviewers a complete story while keeping external configuration manageable.

1. Configure production services.

Configure production services. Add Firebase, GraphHopper, Twilio, Google Places, and OpenAI values as server environment variables. Keep OPENAI_API_KEY in .env.local locally and Vercel server settings in deployment.

2. Validate with realistic scenarios.

Validate with realistic scenarios. Test keyboard navigation, mobile layout, Malayalam text, denied location permissions, missing integrations, invalid image uploads, and emergency fallback messages before a demo.

3. Manage AI usage deliberately.

Manage AI usage deliberately. Use the Responses API only after a user requests assistant help, scans an image, prepares a report, starts indoor guidance, or asks for emergency wording. Keep short inputs, structured responses, and local fallbacks.

Conclusion

The Sahaya prototype has a coherent service boundary and a feature set that maps to high-impact wheelchair-accessibility tasks. Its strongest next step is field validation with representative users and local accessibility stakeholders.

The platform should continue to favor practical, plain-language support over overconfident automation. This approach supports both accessibility and trust.

Appendix

Implementation notes

Client routes include login, registration, navigation, indoor navigation, scanner, community reports, emergency SOS, and the assistant. Feature routes are guarded through the shared authentication shell.

OpenAI is server-only. Assistant and scanner routes use the Responses API; community, indoor, and emergency routes request compact structured text only when AI interpretation adds value.

Source notes

Sahaya project source code and configuration, reviewed July 2026.

Sahaya README: environment variables, deployment notes, and OpenAI cost controls.

Next.js production build output: compiled successfully, type validation passed, and 22 application routes generated.

OpenAI developer documentation: Responses API text and vision guidance.

Report scope. This design report is based on the current project implementation and build verification. It does not certify physical accessibility, regulatory compliance, or live third-party service availability.